Lecture 6: Number Systems and Binary-Decimal Conversions

1. Core Mathematical Paradigm: Building Numbers

When converting between number systems programmatically (without using arrays or strings), we must construct the final integer dynamically using powers of 10.

- **Forward Assembly (Right-to-Left Construction):** To append a new digit to the *left* of an existing number, use the formula:

$$Answer = (digit \times 10^{i}) + Answer$$

(Where i starts at 0 and increments).

- **Reverse Assembly (Left-to-Right Construction):** To append a new digit to the *right* of an existing number, use the formula:

$$Answer = (Answer \times 10) + digit$$

Problem-Solving Deconstruction

Problem 1: Decimal to Binary Conversion (Positive Integers)

1. Problem Statement & Constraints: Given a positive base-10 integer N, convert it into its base-2 (binary) integer representation.

2. Core Intuition: Instead of the traditional mathematical approach (dividing by 2 and storing remainders), we can leverage optimal bitwise operators. The expression `N & 1` reliably extracts the Least Significant Bit (LSB). After extraction, we logically right-shift the number (`N >> 1`) to queue up the next bit. We use the **Forward Assembly** formula to construct the binary representation mathematically.

3. Algorithmic Steps:

1. Initialize `ans = 0` and position multiplier `i = 0`.
2. Open a `while` loop that continues as long as `n != 0`.
3. Extract the current bit: `int bit = n & 1`.
4. Construct the answer: `ans = (bit * pow(10, i)) + ans`.
5. Shift N to the right to discard the processed bit: `n = n >> 1`.
6. Increment `i` to scale the next power of 10.
7. Print the final `ans`.

4. Dry Run:

Input: $N = 5$ (Binary `101`)

1. `i=0, ans=0`. Bit = `5 & 1` = 1. `ans = (1 * 10^0) + 0 = 1`. $N \gg 1 \to 2$.
2. `i=1, ans=1`. Bit = `2 & 1` = 0. `ans = (0 * 10^1) + 1 = 1`. $N \gg 1 \to 1$.
3. `i=2, ans=1`. Bit = `1 & 1` = 1. `ans = (1 * 10^2) + 1 = 101`. $N \gg 1 \to 0$. Loop breaks. *Output:* `101`.

5. Complexity Analysis Table:

Metric	Complexity	Justification
Time Complexity	$O(\log_2 N)$	The loop executes once for every bit in N. The number of bits is $\approx \log_2(N)$.
Space Complexity	$O(1)$	Constant allocation for `ans`, `i`, and `bit`.

C++

```
#include<iostream>
#include<math.h>
using namespace std;

int main() {
    int n;
    cin >> n;

    int ans = 0;
    int i = 0;

    while(n != 0) {
        int bit = n & 1; // Extract LSB

        // Forward Assembly formula
        ans = (bit * pow(10, i)) + ans;

        n = n >> 1;      // Right shift to process next bit
        i++;
    }

    cout << "Answer is " << ans << endl;
    return 0;
}
```

Problem 2: Decimal to Binary Conversion (Negative Integers)

1. Problem Statement & Constraints: Given a negative base-10 integer N, print its binary representation. *Constraint constraint:* Computers natively store negative numbers in 2's complement.

2. Core Intuition: If we simply right-shift a negative number, the compiler may pad it with 1s (arithmetic shift), causing an infinite loop

or incorrect extraction. To safely extract the 2's complement representation up to a specific bit limit (e.g., 16 bits), we can simulate the 2's complement mathematically: $2^{16} - |N|$ is equivalent to adding N (since N is negative) to 2^{16}. This casts the negative number into an unsigned positive space that perfectly matches the binary bits of the negative 2's complement.

3. Algorithmic Steps: 1. Initialize variables. Crucially, use `unsigned long long int` for the answer to prevent overflow during bit construction.

2. If $N < 0$, mathematically enforce the 16-bit 2's complement: `n = pow(2, 16) + n`.
3. Process the resulting positive N exactly like Problem 1 using `while(n != 0)`.
4. Extract bits with `n & 1`, assemble with `pow(10, i)`, and shift right `n >> 1`.

4. Complexity Analysis Table:

Metric	Complexity	Justification
Time Complexity	$O(K)$	Where K is the fixed bit-width (e.g., 16 or 32). Effectively $O(1)$ constant time.
Space Complexity	$O(1)$	Utilizes standard scalar variables.

C++

```
#include<iostream>
#include<math.h>
using namespace std;

int main() {
    long long int n;
    unsigned long long int i = 0, ans = 0; // Prevent overflow on 16-bit
integers

    cout << "Decimal Number : ";
    cin >> n;

    // Simulate 16-bit 2's complement conversion
    if (n < 0) {
        n = pow(2, 16) + n;
    }

    while (n != 0) {
        int bit = n & 1;
        ans = bit * pow(10, i) + ans;
        n = n >> 1;
```

```
        i++;
    }

    cout << "Binary Number : " << ans << endl;
    return 0;
}
```

Problem 3: Binary to Decimal Conversion

1. Problem Statement & Constraints: Given a base-2 (binary) integer represented sequentially in base-10 (e.g., the `int` 101 representing binary 101), convert it back to a true base-10 decimal integer.

2. Core Intuition: We must parse the input number digit by digit from right to left. In base-10 math, we extract the rightmost digit using `n % 10` and discard it using `n / 10`. If the extracted digit is a 1, we calculate its actual base-10 weight using 2^i and add it to our accumulator.

3. Algorithmic Steps: 1. Initialize `ans = 0` and position multiplier `i = 0`.

2. Open a `while` loop bounded by `n != 0`.
3. Extract the rightmost digit: `digit = n % 10`.
4. Check condition: If `digit == 1`, add 2^i to `ans` (`ans = ans + pow(2, i)`). If it is 0, we do nothing (since $0 \times 2^i = 0$).
5. Divide N by 10 (`n = n / 10`) to queue the next digit.
6. Increment i and repeat.

4. Dry Run:

Input: $N = 101$

1. `i=0, ans=0`. Digit = `101 % 10` = 1. Digit is $1 \rightarrow$ `ans = 0 + 2^0 = 1`. $N/10 \rightarrow 10$.
2. `i=1, ans=1`. Digit = `10 % 10` = 0. Digit is $0 \rightarrow$ ignore. $N/10 \rightarrow 1$.
3. `i=2, ans=1`. Digit = `1 % 10` = 1. Digit is $1 \rightarrow$ `ans = 1 + 2^2 = 5`. $N/10 \rightarrow 0$. Loop breaks.

Output: 5.

5. Complexity Analysis Table:

Metric	Complexity	Justification
Time Complexity	$O(\log_{10} N)$	The loop executes exactly once for every digit in the pseudo-binary base-10 integer.
Space Complexity	$O(1)$	Variables `ans`, `i`, and `digit` utilize constant space.

C++

```
#include<iostream>
#include<math.h>
using namespace std;

int main() {
    int n;
    cin >> n;

    int i = 0, ans = 0;

    while(n != 0) {
        int digit = n % 10; // Extract rightmost digit mimicking binary
structure

        // Only calculate power if bit is 1 (optimization)
        if(digit == 1) {
            ans = ans + pow(2, i);
        }

        n = n / 10; // Discard processed digit
        i++;
    }

    cout << ans << endl;
    return 0;
}
```